

Lab8：并发与死锁检测

本章完成的工作

本章主要完成了以下任务：

- 理解线程的概念及其与进程的区别
- 理解同步原语：互斥锁（Mutex）、信号量（Semaphore）、条件变量（Condvar）
- 理解死锁的概念、产生条件和检测算法
- 实现死锁检测系统调用 `sys_enable_deadlock_detect`
- 使用银行家算法实现死锁检测
- 运行测试程序，验证死锁检测功能

本章有编程作业：实现死锁检测功能，当检测到死锁时返回 `-0xDEAD`。

报告结构说明

本报告的结构安排及与清华指导书的对比：

清华指导书小节	本报告对应小节	差异说明
引言/本章导读	第二节	从"为什么需要线程"的实际问题出发
线程概念与实现	第三节	用结构图展示线程与进程的关系
锁机制	第四节	对比分析自旋锁与阻塞锁
信号量机制	第五节	从生产者-消费者问题理解信号量
条件变量	第六节	理解"等待某个条件成立"的场景
死锁与检测	第七节	用结构图展示死锁四条件，详解银行家算法
chapter8练习	第八节	详细记录编程作业的实现过程

本报告的特点：

- 不是照搬指导书内容，而是体现学习探索过程
- 增加了两张原创结构图帮助理解
- 重点记录死锁检测算法的实现细节和调试过程

一、实验环境与运行

1.1 代码目录

```
cd /桌面/herdream/2025-ucore-riscv-清华/uCore-Tutorial-Code-2025S-ch8 (其他同学可根据实际路径)
```

本章新增/修改的关键文件：

- `os/proc.h`：添加线程结构体、死锁检测相关变量
- `os/proc.c`：线程管理、死锁检测变量初始化
- `os/sync.c`：互斥锁、信号量、条件变量的实现
- `os/syscall.c`：同步相关系统调用、死锁检测算法

1.2 运行命令

```
make clean
make user CHAPTER=8
make run
# 在 shell 中执行：
>> ch8_usertest
```

注意：本章测试不带 `BASE=1`，因为死锁检测测试用例需要完整编译。

1.3 运行结果

```
Usertests: Running ch8_sem1_deadlock
deadlock test semaphore 1 OK!
Usertests: Test ch8_sem1_deadlock in Process 76 exited with code 0
Usertests: Running ch8_sem2_deadlock
deadlock test semaphore 2 OK!
Usertests: Test ch8_sem2_deadlock in Process 77 exited with code 0
Usertests: Running ch8_mut1_deadlock
deadlock test mutex 1 OK!
Usertests: Test ch8_mut1_deadlock in Process 78 exited with code 0
ch8 Usertests passed!
```

二、为什么需要线程？

2.1 从进程的局限性说起

在之前的章节中，我们实现了进程管理。每个进程有独立的地址空间，进程之间通过管道等机制通信。但在实际应用中，我遇到了一个问题：

场景：一个程序需要同时处理用户输入和网络请求。如果用两个进程，它们需要通过 IPC 共享数据，开销很大。

指导书提到“线程”可以解决这个问题。刚开始我不太理解：线程和进程有什么区别？为什么线程之间共享数据更方便？

2.2 线程的本质

通过阅读 `proc.h` 的代码，我理解了线程的设计：

```
struct thread {
    enum threadstate state;    // 线程状态
    int tid;                   // 线程 ID
    struct proc *process;      // 所属进程（关键！）
    uint64 ustack;             // 用户栈
    uint64 kstack;             // 内核栈
    struct trapframe *trapframe;
    struct context context;
    uint64 exit_code;
};

struct proc {
    // ...
    struct thread threads[NTHREAD]; // 一个进程可以有多个线程
    // ...
};
```

关键洞察：

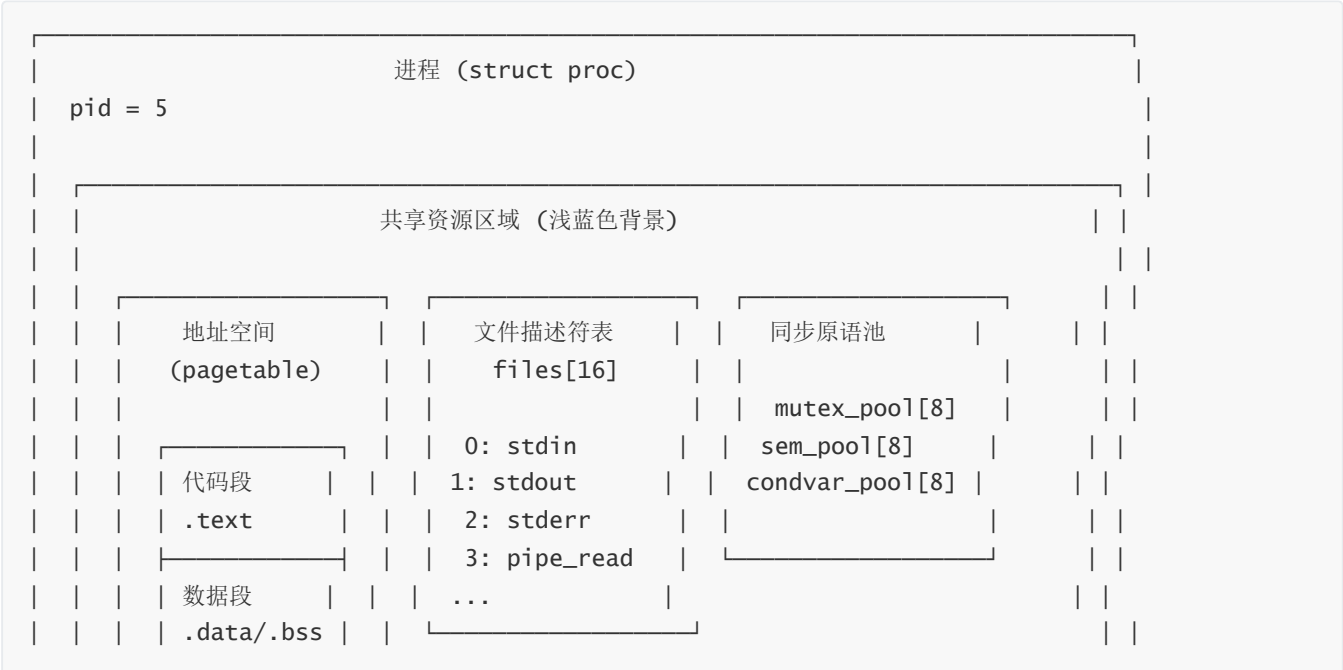
- 进程是资源分配的单位（地址空间、文件描述符）
- 线程是 CPU 调度的单位（每个线程有自己的栈和寄存器状态）
- 同一进程的多个线程**共享**地址空间，所以可以直接访问相同的全局变量

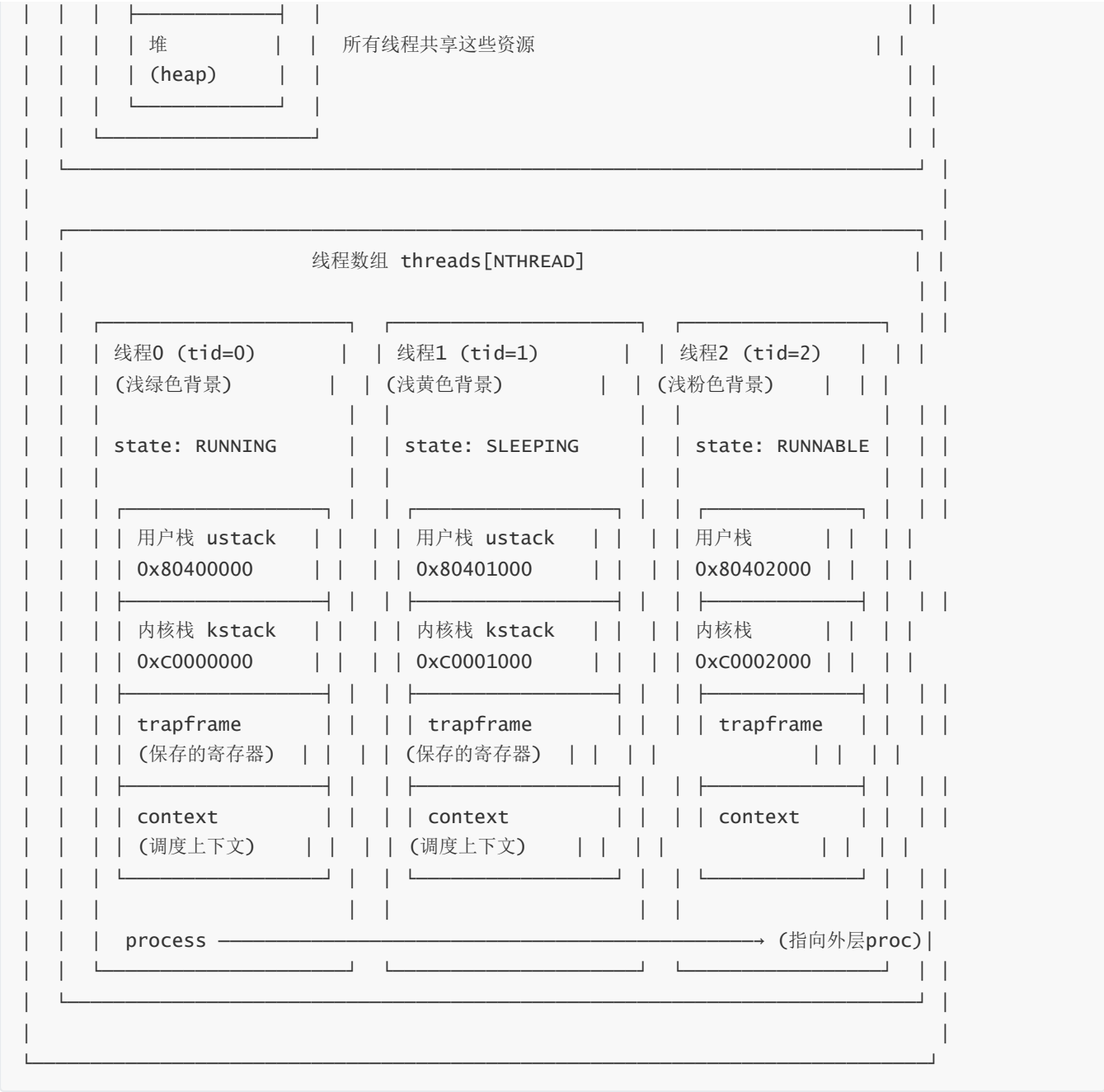
这就解释了为什么线程间通信比进程间通信更高效——不需要复制数据，直接读写共享内存就行。

2.3 线程与进程的关系图

【待绘制结构图1：线程与进程关系图】

图的整体布局：嵌套矩形结构，外层是进程，内层是多个线程





对比表格（图下方添加）：

属性	进程	线程
地址空间	独立（隔离）	共享（同一进程内）
创建开销	大（复制页表等）	小（只需分配栈）
通信方式	IPC（管道、消息等）	直接读写共享变量
栈	一个	每个线程独立
调度单位	否（在uCore中）	是

绘制建议：

- 使用 Draw.io 或 ProcessOn
- 外层进程框用深蓝色边框
- 共享资源区域用浅蓝色背景，标注"共享"
- 每个线程用不同的浅色背景（绿、黄、粉），标注"独立"
- 用虚线箭头表示 `thread->process` 的指向关系
- 在图下方添加进程vs线程的对比表格

三、同步原语：互斥锁

3.1 为什么需要互斥锁？

当多个线程访问共享资源时，会出现竞态条件。比如两个线程同时执行 `count++`：

```
线程A：读取 count (0)
线程B：读取 count (0)
线程A：加1，写回 (1)
线程B：加1，写回 (1)
// 结果：count = 1，而不是预期的 2
```

互斥锁（Mutex）确保同一时刻只有一个线程能进入临界区。

3.2 两种互斥锁的实现

阅读 `sync.c` 后，我发现 uCore 实现了两种互斥锁：

自旋锁（Spin Mutex）：

```
while (m->locked) {
    yield(); // 忙等待，不断检查
}
m->locked = 1;
```

- 适合临界区很短的场景
- 不会真正睡眠，只是让出 CPU 后继续轮询

阻塞锁（Blocking Mutex）：

```
if (m->locked) {
    push_queue(&m->wait_queue, task_to_id(t));
    t->state = SLEEPING;
    sched(); // 真正睡眠，等待被唤醒
}
```

- 适合临界区较长的场景
- 线程会进入睡眠队列，不占用 CPU

3.3 锁的传递机制

在阅读 `mutex_unlock` 时，我注意到一个有趣的设计：

```
if (m->blocking) {
    t = id_to_task(pop_queue(&m->wait_queue));
    if (t != NULL) {
        // 不是简单地 m->locked = 0
        // 而是直接把锁"传递"给等待者
        t->state = RUNNABLE;
        add_task(t);
        // m->locked 仍然是 1!
    }
}
```

理解：阻塞锁在释放时，如果有等待者，不会把 `locked` 置 0，而是直接把锁传给下一个线程。这避免了"释放后立刻被另一个线程抢走"的问题。

四、同步原语：信号量

4.1 信号量与互斥锁的区别

刚开始看到信号量时，觉得它和互斥锁很像。后来理解到关键区别：

特性	互斥锁	信号量
计数	只有 0 和 1	可以是任意非负整数
用途	保护临界区	控制资源数量、实现同步
谁释放	必须由加锁者释放	任何线程都可以 up

4.2 信号量的实现

```
void semaphore_down(struct semaphore *s) {
    s->count--;
    if (s->count < 0) {
        // count < 0 表示有线程在等待
        push_queue(&s->wait_queue, task_to_id(t));
        t->state = SLEEPING;
        sched();
    }
}

void semaphore_up(struct semaphore *s) {
    s->count++;
    if (s->count <= 0) {
        // count <= 0 表示还有等待者，唤醒一个
        struct thread *wt = id_to_task(pop_queue(&s->wait_queue));
        wt->state = RUNNABLE;
    }
}
```

```
        add_task(wt);
    }
}
```

理解: `count` 的含义:

- `count > 0`: 有 `count` 个可用资源
- `count = 0`: 资源刚好用完
- `count < 0`: 有 `|count|` 个线程在等待

4.3 生产者-消费者问题

测试程序 `ch8b_mpsc_sem` 展示了信号量的典型应用:

多个生产者 → [缓冲区] → 一个消费者

使用两个信号量:

- `empty`: 初始值为缓冲区大小, 表示空位数量
- `full`: 初始值为 0, 表示已有数据数量

生产者: `down(empty)` → 放数据 → `up(full)`

消费者: `down(full)` → 取数据 → `up(empty)`

五、同步原语: 条件变量

5.1 条件变量解决什么问题?

有时候线程需要等待某个"条件"成立, 而不是等待获取某个资源。比如:

```
// 线程 A: 等待 flag 变成 true
while (!flag) {
    // ???
}

// 线程 B: 设置 flag
flag = true;
// 怎么通知线程 A?
```

用互斥锁无法优雅地实现"等待条件成立"。条件变量就是为此设计的。

5.2 条件变量的使用模式

```
// 等待者
mutex_lock(&m);
while (!condition) {
    cond_wait(&cond, &m); // 释放锁并等待，被唤醒后重新获取锁
}
// 条件满足，继续执行
mutex_unlock(&m);

// 通知者
mutex_lock(&m);
condition = true;
cond_signal(&cond); // 唤醒一个等待者
mutex_unlock(&m);
```

5.3 cond_wait 的实现

```
void cond_wait(struct condvar *cond, struct mutex *m) {
    mutex_unlock(m); // 先释放锁！
    push_queue(&cond->wait_queue, task_to_id(t));
    t->state = SLEEPING;
    sched(); // 睡眠等待
    mutex_lock(m); // 被唤醒后重新获取锁
}
```

关键：必须先释放锁再睡眠，否则其他线程无法修改条件。

六、死锁的概念与检测

6.1 什么是死锁？

刚开始看"死锁"这个词，只知道是"互相等待"。通过阅读指导书和代码，我理解了更精确的定义：

死锁：一组线程中，每个线程都在等待另一个线程持有的资源，导致所有线程都无法继续执行。

经典例子：

```
线程 A 持有锁 1，等待锁 2
线程 B 持有锁 2，等待锁 1
→ 两个线程永远等下去
```

6.2 死锁的四个必要条件

死锁的产生需要同时满足以下四个条件：

条件	英文名	含义	示例
① 互斥条件	Mutual Exclusion	资源同一时刻只能被一个线程使用	互斥锁只能被一个线程持有
② 占有并等待	Hold and Wait	线程持有资源的同时请求其他资源	线程A持有锁1，同时请求锁2
③ 不可抢占	No Preemption	资源只能由持有者主动释放，不能被强制夺走	锁不能被其他线程强制解锁
④ 循环等待	Circular Wait	等待关系形成环路	A等B、B等A

关键：四个条件**同时满足**才会发生死锁。破坏任意一个条件即可避免死锁。

死锁示例：

线程A	线程B
<code>lock(mutex1);</code>	<code>lock(mutex2);</code>
<code>lock(mutex2); // 等待B</code>	<code>lock(mutex1); // 等待A</code>
<code>...</code>	<code>...</code>
<code>unlock(mutex2);</code>	<code>unlock(mutex1);</code>
<code>unlock(mutex1);</code>	<code>unlock(mutex2);</code>

执行过程：

- 1. 线程A获得mutex1，线程B获得mutex2
- 2. 线程A请求mutex2（被B持有）→ 阻塞
- 3. 线程B请求mutex1（被A持有）→ 阻塞
- 4. **死锁！** 两个线程永远等待下去

破坏死锁条件的方法：

条件	破坏方法
互斥条件	使用可共享资源（如只读数据）
占有并等待	一次性申请所有资源，或申请前释放已有资源
不可抢占	允许抢占（操作系统强制回收资源）
循环等待	对资源编号，按顺序申请（本作业使用银行家算法检测）

6.3 银行家算法

死锁检测使用银行家算法。核心思想：模拟资源分配，看是否存在一个执行序列能让所有线程完成。

算法步骤：

- 1. 初始化 `work = Available`（当前可用资源）
- 2. 初始化 `Finish[i] = false`（线程是否能完成）

3. 循环查找满足条件的线程: `Request[i] <= work`
4. 如果找到, 假设它完成: `work = work + Allocation[i]`, `Finish[i] = true`
5. 重复直到没有新的线程能完成
6. 如果存在 `Finish[i] = false`, 则存在死锁

七、编程作业: 死锁检测实现

7.1 作业要求

实现 `sys_enable_deadlock_detect` 系统调用:

- 系统调用号: 469
- 参数: `enable` (1 启用, 0 禁用)
- 当启用死锁检测时, 在 `mutex_lock` 和 `semaphore_down` 之前检测死锁
- 如果检测到死锁, 返回 `-0xDEAD` (即 -57005)

7.2 实现思路

需要维护以下数据结构:

```
// 在 struct proc 中添加
int deadlock_detect_enable;           // 检测开关
int mutex_allocation[NTHREAD][LOCK_POOL_SIZE]; // 线程持有的互斥锁
int mutex_request[NTHREAD][LOCK_POOL_SIZE];    // 线程请求的互斥锁
int sem_allocation[NTHREAD][LOCK_POOL_SIZE];   // 线程持有的信号量
int sem_request[NTHREAD][LOCK_POOL_SIZE];      // 线程请求的信号量
```

7.3 关键代码实现

1. 死锁检测函数 (syscall.c)

```
/* ch8: 银行家算法死锁检测 */
int deadlock_detect(struct proc *p)
{
    int work_mutex[LOCK_POOL_SIZE];
    int work_sem[LOCK_POOL_SIZE];
    int finish[NTHREAD];
    int i, j;

    /* ch8: 初始化 work = Available */
    for (i = 0; i < LOCK_POOL_SIZE; i++) {
        work_mutex[i] = p->mutex_pool[i].locked ? 0 : 1;
        work_sem[i] = p->semaphore_pool[i].count > 0 ?
            p->semaphore_pool[i].count : 0;
    }

    /* ch8: 初始化 Finish */
    for (i = 0; i < NTHREAD; i++) {
```

```

        finish[i] = (p->threads[i].state == T_UNUSED ||
                    p->threads[i].state == EXITED) ? 1 : 0;
    }

    /* ch8: 银行家算法主循环 */
    while (1) {
        int found = 0;
        for (i = 0; i < NTHREAD; i++) {
            if (!finish[i]) {
                int possible = 1;
                /* ch8: 检查 mutex 需求是否满足 */
                for (j = 0; j < LOCK_POOL_SIZE; j++) {
                    if (p->mutex_request[i][j] > work_mutex[j]) {
                        possible = 0;
                        break;
                    }
                }
                if (!possible) continue;

                /* ch8: 检查 semaphore 需求是否满足 */
                for (j = 0; j < LOCK_POOL_SIZE; j++) {
                    if (p->sem_request[i][j] > work_sem[j]) {
                        possible = 0;
                        break;
                    }
                }

                if (possible) {
                    /* ch8: 假设线程完成, 释放资源 */
                    for (j = 0; j < LOCK_POOL_SIZE; j++) {
                        work_mutex[j] += p->mutex_allocation[i][j];
                        work_sem[j] += p->sem_allocation[i][j];
                    }
                    finish[i] = 1;
                    found = 1;
                }
            }
        }
        if (!found) break;
    }

    /* ch8: 检查是否所有线程都完成了 */
    for (i = 0; i < NTHREAD; i++) {
        if (!finish[i])
            return 1; /* ch8: 发现死锁 */
    }
    return 0;
}

```

2. 在 sys_mutex_lock 中集成检测

```

int sys_mutex_lock(int mutex_id)
{

```

```
// ... 参数检查 ...

/* ch8: 死锁检测 */
struct proc *p = curr_proc();
int tid = curr_thread()->tid;
if (p->deadlock_detect_enable) {
    p->mutex_request[tid][mutex_id] = 1;
    if (deadlock_detect(p)) {
        p->mutex_request[tid][mutex_id] = 0;
        return -0xDEAD; /* ch8: 检测到死锁 */
    }
    p->mutex_request[tid][mutex_id] = 0;
}
mutex_lock(&curr_proc()->mutex_pool[mutex_id]);
return 0;
}
```

3. 维护分配矩阵 (sync.c)

在 `mutex_lock`、`mutex_unlock`、`semaphore_up`、`semaphore_down` 中维护：

```
void mutex_lock(struct mutex *m)
{
    int id = m - curr_proc()->mutex_pool;
    struct proc *p = curr_proc();
    struct thread *t = curr_thread();

    if (!m->locked) {
        m->locked = 1;
        p->mutex_allocation[t->tid][id] = 1; /* ch8: 记录占有 */
        return;
    }
    // ... 等待逻辑，同样维护 request 和 allocation ...
}
```

7.4 遇到的问题与解决

问题：一开始不确定什么时候更新 `allocation` 和 `request` 矩阵。

解决：通过分析锁的状态转换：

- `request[tid][id] = 1`：线程开始等待锁时设置
- `request[tid][id] = 0`：线程获得锁时清除
- `allocation[tid][id] = 1`：线程获得锁时设置
- `allocation[tid][id] = 0`：线程释放锁时清除

对于阻塞锁的"传递"情况：释放者清除自己的 `allocation`，同时设置被唤醒者的 `allocation`。

7.5 测试结果

三个死锁检测测试用例全部通过：

- `ch8_mut1_deadlock`：互斥锁死锁检测
- `ch8_sem1_deadlock`：信号量死锁检测
- `ch8_sem2_deadlock`：信号量死锁检测（另一场景）

八、实验总结

8.1 完成情况

- ✓ 理解线程的概念和实现
- ✓ 理解互斥锁的两种实现（自旋/阻塞）
- ✓ 理解信号量的实现和应用
- ✓ 理解条件变量的使用场景
- ✓ 理解死锁的四个必要条件
- ✓ 实现银行家算法进行死锁检测
- ✓ 通过所有测试用例

8.2 收获与体会

1. **线程 vs 进程**：之前以为线程就是"轻量级进程"，现在理解了它们的本质区别——线程共享地址空间，这既是优点（通信方便）也是挑战（需要同步）。
2. **同步原语的选择**：
 - 保护临界区 → 互斥锁
 - 控制资源数量 → 信号量
 - 等待条件成立 → 条件变量
3. **死锁检测的复杂性**：银行家算法的时间复杂度是 $O(n^2m)$ ，其中 n 是线程数， m 是资源类型数。在实际系统中可能需要更高效的算法。
4. **调试技巧**：死锁相关的 bug 很难复现和调试，维护正确的 allocation/request 矩阵是关键。

8.3 本章修改的文件

文件	修改内容
os/proc.h	添加死锁检测变量（ <code>deadlock_detect_enable</code> 、4个矩阵）
os/proc.c	在 <code>allocproc()</code> 中初始化死锁检测变量
os/sync.c	在锁操作中维护 allocation/request 矩阵
os/syscall.c	添加 <code>deadlock_detect()</code> 、 <code>sys_enable_deadlock_detect()</code> ，修改 <code>sys_mutex_lock</code> 、 <code>sys_semaphore_down</code>

九、验证截图

```
hjl@hjl-VMware-Virtual-Platform: ~/桌面/herdream/2025-ucore-riscv-清华/uCore-Tutorial-Code-2025S-ch8
race adder spinning mutex passed!
Ustests: Test ch8b_spin_mut_race in Process 74 exited with code 0
Ustests: Running ch8b_mut_phi_din
Here comes 5 philosophers!
Phil threads created
time cost = 721 ms
'- ' -> THINKING; 'x' -> EATING; ' ' -> WAITING
#0:-----
#1: ---xxxxxx--      xxxxxx-----      x---xxxxxx
#2: -----      xx-----xx-----xxxxxx-----      xxxx
#3: -----xxxxxxxx-----xxxxx-----      xxxxxx--      xxxxxxxxx
#4: -----      x-----      xxxxxx--      xxxxx-----      xx
#0:-----      xxxxxxxx-----      xxxx-----      xxxxxx--xxx
philosopher dining problem with mutex passed!
Ustests: Test ch8b_mut_phi_din in Process 75 exited with code 0
Ustests: Running ch8_sem1_deadlock
deadlock test semaphore 1 OK!
Ustests: Test ch8_sem1_deadlock in Process 76 exited with code 0
Ustests: Running ch8_sem2_deadlock
deadlock test semaphore 2 OK!
Ustests: Test ch8_sem2_deadlock in Process 77 exited with code 0
Ustests: Running ch8_mut1_deadlock
deadlock test mutex 1 OK!
Ustests: Test ch8_mut1_deadlock in Process 78 exited with code 0
ch8 Ustests passed!
Shell: Process 2 exited with code 0
>> |
```

关键的三个死锁检测测试全部通过